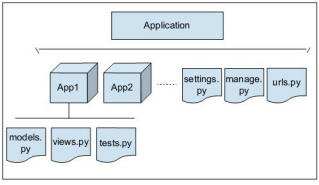

Figure 1: Folder structure of a Django application

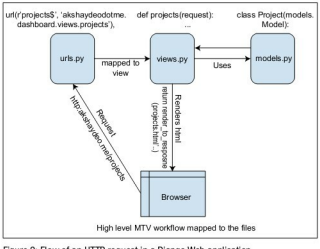

Figure 2: Flow of an HTTP request in a Django Web application

wide array of database servers, but you won't have to deal directly with them; Django provides a unified layer of access to all available database engines.

3. ***MTV architecture:*** MTV architecture: M stands for 'Model', the data access layer, which contains everything about the data—how to access it, how to validate it, its characteristics, and the relationships between data. T stands for 'Template', the presentation layer, which contains presentation-related decisions—how something should be displayed on a Web page. These are HTML files with some Django tags which are processed while being rendered. V stands for 'View', which basically acts as controller which renders an HTML page using templates and models.
4. ***A Robust ORM system:*** Django has a powerful Object-Relational Mapping system. It supports a large set of database systems, and switching from one engine to another is a matter of changing a configuration file. This gives the developer great flexibility if a decision is made to change from one database engine to another.
5. ***Beautiful URLs:*** Compare a URL from any Django website to those of an ASP or PHP site, and you'll notice a big difference. You can read the Django URL. And that's good for you, and Google, since it's factored into the page's importance. Some Web frameworks come close to Django in URL construction. But what makes Django's URLs extra-nice is the ease with which you can create them.
6. ***A powerful template system:*** Django has a very powerful template system to inject programming logic inside templates. The framework is great at simplifying complex things, and the template system is no exception. It is clear and easy to adopt, which speeds up development.
7. ***Automatic admin interface:*** Django comes up with a ready-made admin interface, which allows a developer to control Web applications via a Web portal. This admin interface is generic, and can be extended and customised, depending upon project requirements.
8. ***Complete development environment:*** Django provides a complete development environment. It comes with a lightweight Web server for development and testing. When the debugging mode is enabled, Django provides very thorough and detailed error messages, with a lot of debugging information—which, in turn, makes debugging easier, and allows one to quickly isolate bugs.
9. ***A rapidly growing community:*** The Django community is growing really fast. A strong community is always helpful when you need some help, training, reusable modules, utility tools, etc. Django has its own packages for almost all types of reusable modules like error logging, automated deployments, social networking integrations, blogging and so on.

Django project architecture (Django 1.3)

An empty Django project folder structure (see Figure 1) contains three important files:

- *settings.py:* This contains project settings, such as database connection information, email server configuration, installed apps, media folder location and static folder location with their URLs, template folder location and reference to some Django modules.
- *urls.py*: Can be considered the 'DNS' of our Django app. Each request to the application goes through *urls.py*, and entries in the *urlpatterns* list in this decide which request is to be forwarded to which view of a particular app.
- *manage.py:* Contains commands that, as the name suggests, are used for managing our Django project—for example, *startapp*, *syncdb*, etc.

Each Django Web application comprises small modules called apps. If we consider Wikipedia as our project, the project would contain *user* as one app, which would handle all functionality related to user management. *Page* would be another app, handling all page-related functionality, and would be used by the *user* app, since each page is related to a user that created or modified it.

Each app inside our project contains three files:

- *models.py:* models.py: This is the place for all your application data (that you want to be persistent). These are basically classes extending models.Model, which is